

An Integrated CFD Workflow Platform for In-House Solvers: Modular Architecture and AI-Assisted Implementation

HsuehJui Lu^{1,*}, ShengJer Chen¹ and FangAn Kuo¹

¹National Center for High-performance Computing, National Institutes of Applied Research,
Taiwan

*hjlu@niar.org.tw

Abstract. To reduce the adoption barrier of in-house solvers, we present an integrated CFD workflow platform that unifies geometry preparation, mesh generation, solver execution, and result visualization in a single environment without modifying the solver core, eliminating manual operations typical of in-house solver workflows. Development used an AI-assisted approach with domain experts retaining full architectural and validation responsibility. Applied to in-house CFD solver UNICONES, the platform reduced required tools from five to one and workflow steps from twenty to five, with onboarding time estimated at two days versus two weeks. The design is generalizable to other in-house solvers.

Keywords: CFD workflow integration; in-house solver; solver adapter; AI-assisted software development; agentic coding.

1. Introduction

In-house CFD solvers often achieve high numerical maturity yet see limited adoption due to workflow complexity. Users must coordinate separate tools, manage manual file conversions, and navigate solver-specific configuration with little GUI support, which shows a significant barrier even when the underlying solver is well validated. UNICONES [1], a compressible Euler/Navier–Stokes solver based on the CESE method [2], developed at the NCHC, National Institutes of Applied Research of Taiwan, is representative, despite open academic availability, familiarization typically requires several weeks of guided effort.

The present work addresses this gap with an integrated platform that unifies the full simulation pipeline without modifying the solver core, using an AI-assisted scientific software development approach [3,4] in which domain experts retain full architectural and validation responsibility.

This paper makes the following contributions (i) a modular solver-adapter interface enabling lightweight integration of heterogeneous solvers that lowers the adoption barrier of in-house solvers for new users; (ii) a documented human–AI collaborative development methodology with empirical observations on contribution distribution; and (iii) a generalizable workflow integration design strategy applicable to research groups maintaining in-house CFD solvers with limited software engineering resources.

2. System Architecture

The platform integrates four modules — PreProcessor, Mesh Generator, Core Solver, and Results Visualization — within a unified PyQt6 GUI covering the complete simulation pipeline (Fig. 1). The preprocessor provides an interactive geometry canvas supporting multiple point-spacing strategies, automatic sharp-corner detection, and per-segment resampling via a standalone C++ backend. The mesh generation core (~1,500 lines C++) produces 2D hybrid meshes, including structured boundary-layer quads via node-advancement [5] and unstructured far-field triangles via the Gmsh Frontal-Delaunay algorithm [6], exported in both VTK and STAR-CD format (.vrt/.cel/.bnd) with all inter-stage conversions handled automatically. The core solver module manages input format conversion, optional MPI domain decomposition, and solver execution, including solvers distributed as shared libraries via a built-in DLL call template, with a background worker parsing stdout in real time for live convergence monitoring. The results visualization module parses Tecplot-format output and renders cell-centered scalar fields on an embedded matplotlib canvas, supporting contour plots, streamlines, and vector glyphs with interactive colormap and field-variable controls.

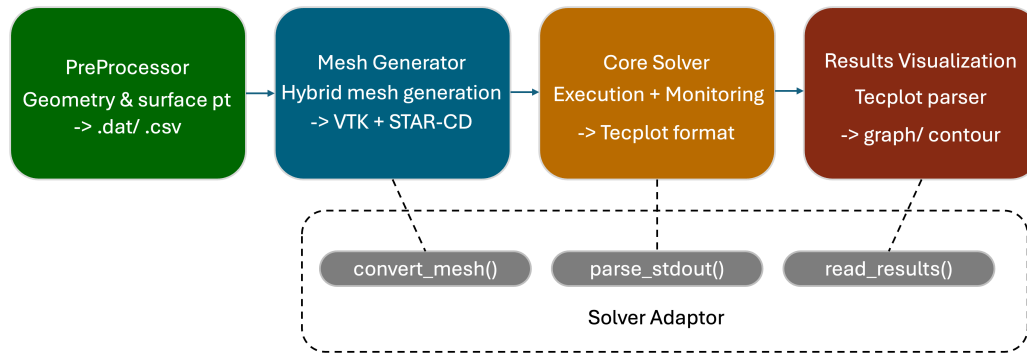


Figure 1. Data flow of the integrated CFD workflow platform.

A Solver Adapter abstraction enables extensibility, as shown in Table 1. Each adapter implements three interface methods while all upstream modules remain unchanged, requiring fewer than 150 lines of Python per new solver.

Table 1. Solver Adapter interface definition.

METHOD	INPUT	OUTPUT	DESCRIPTION
<code>convert_mesh(src, dst)</code>	VTK mesh path	Solver-format mesh file(s)	Converts platform mesh to solver input format (e.g., STAR-CD .vrt/.cel/.bnd for UNICONES).
<code>parse_stdout(line) → dict</code>	One line of solver stdout	Dict of convergence quantities or None	Extracts iteration, CFL, and residual values for live monitoring; returns None for non-data lines.
<code>read_results(path) → dataset</code>	Solver result file path	Structured field dataset (coordinates + scalar fields)	Parses solver-specific output format for the visualization module; fully decouples visualization from solver output format.

3. AI-Assisted Development Methodology

3.1. The Human-AI Collaboration Model

The platform was built using LLM coding assistants in agentic mode, which can directly read, write, and execute project files, beyond purely conversational generation. Domain experts defined the pipeline architecture, numerical algorithms, format-compatibility chain, and physical validation; the AI generated GUI boilerplate, file parsers, and C++ routines from algorithmic descriptions. Each feature typically converged in 2–5 rounds of describe–generate–execute–validate–correct iteration.

3.2. Representative Development Observations

Development of the boundary-layer generation module illustrates the three layers of human oversight that proved indispensable throughout the project. The domain expert described the outward node-advancement algorithm in natural language and the AI generated an initial C++ implementation, but text description alone was insufficient to resolve geometric failures. The AI required actual runtime output to identify self-intersections at sharp trailing edges and incorrect fan-node counts at convex corners. More carefully phrased prompts consistently reduced correction rounds, and even after the AI confirmed a fix as correct, physical validation by the domain expert was still needed to catch discrepancies that emerged only in real use.

The 69-file, 18,000-line PyQt6 GUI application and the solver pipeline integration reinforced the same pattern. AI was highly productive for boilerplate, including widget layouts, signal-slot wiring, and stdout parsers, but workflow-level decisions required explicit human specification, such as mode-switching behavior, cross-panel consistency, and the handling of IBM vs. non-IBM solver output modes that were discovered only during live testing to produce different stdout structures.

3.3. Development Statistics

Table 2 summarizes key development metrics derived from developer logs, reported as case-study observations rather than controlled experimental results. AI contribution was unevenly distributed: highest for GUI boilerplate and file I/O (~80–90%), lowest for domain-specific numerical logic and cross-component integration (~40–60%).

Table 2. AI-assisted development process statistics (case study observations).

METRIC	VALUE	NOTES
Estimated AI-generated code proportion	~70–80%	By line count; higher for GUI/boilerplate, lower for numerical algorithms and integration logic.
Median correction rounds per feature	2–5 rounds	Depends on feature complexity and prompt precision.
Estimated total development effort	~4–6 weeks	Compared to estimated 3–6 months for conventional solo development of equivalent scope.
Primary developer headcount	1 domain expert	No dedicated GUI software engineer.

4. Workflow Evaluation

Compared to using UNICONES without a unified interface, which requires approximately five separate tools and around twenty manual steps per simulation, the integrated platform consolidates the entire workflow into a single application with approximately five steps. Time for new users to become familiar with the full simulation workflow was estimated to decrease from around two weeks to two days, based on informal observation. These figures reflect the elimination of manual format conversions and inter-tool data transfers at each pipeline stage, and should be treated as indicative rather than controlled experimental results.

5. Discussion

The AI-assisted approach proved most effective for peripheral tooling, such as GUI interfaces, data pipelines, and visualization, where LLM strengths in well-established software patterns apply. Effective collaboration required three layers of human oversight: precise problem description, execution-based diagnosis of edge cases, and physical validation. Without domain expertise at each layer, AI-assisted scientific software development risks producing plausible but incorrect implementations.

The Solver Adapter design decouples solver-specific logic from the platform core, which is a new solver requires only three interface methods (fewer than 150 lines of Python) while all upstream modules remain unchanged. This lightweight integration burden, combined with the AI-assisted development approach, suggests the pattern is broadly reproducible by domain-expert teams without dedicated software engineering resources.

6. Conclusion

We presented an integrated CFD workflow platform for in-house solvers to reduce the adoption barrier, demonstrated with UNICONES and built via an AI-assisted scientific software development approach. Without modifying the solver core, the platform reduces required tools from five to one, workflow steps from approximately twenty to five, and estimated onboarding time from two weeks to two days. The modular solver-adaptor architecture, requiring fewer than 150 lines of Python per new solver, provides a generalizable strategy for improving in-house solver accessibility in research groups with limited software engineering resources.

Future work includes extending the platform to 3D mesh generation, conducting a formal user study with standardized usability metrics, and exploring web-based deployment to further lower the barrier for CFD education and research.

Acknowledgement

This study was made possible by the academic resources and advanced research infrastructure provided by the National Center for High-performance Computing, National Institutes of Applied Research of Taiwan. The research and development of the software framework was partially funded by the fundamental R&D program provided by NSTC of Taiwan. We gratefully acknowledge their invaluable support.

References

- [1] Lu, H., Chen, K. L., Chang, C. L., & Cheng, G. C. (2026). Interpolation-Free Immersed Boundary Method Implementation Using the Space-Time Conservation Framework in UNICONES. In AIAA SCITECH 2026 Forum (p. 2525).
- [2] Chang, S. C. (1995). The method of space-time conservation element and solution element—A new approach for solving the Navier-Stokes and Euler equations. *Journal of computational Physics*, 119(2), 295-324.
- [3] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., ... & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), 1-79.
- [4] Beaulieu, N., Dascalu, S., & Hand, E. (2025, June). Vibe Coding: A Multivocal Systematic Mapping Study. In 2025 IEEE/ACIS 29th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD) (pp. 266-273). IEEE.
- [5] Thompson, J. F., Soni, B. K., & Weatherill, N. P. (Eds.). (1998). *Handbook of grid generation*. CRC press.
- [6] Geuzaine, C., & Remacle, J. F. (2009). Gmsh: A 3-D finite element mesh generator with built-in pre-and post-processing facilities. *International journal for numerical methods in engineering*, 79(11), 1309-1331.